非线性函数工具集技术说明

1. 技术方案原理

nonlinear_functions 测试由基础协议和混合近似组合而成的高阶神经网络非线性算子，覆盖 GeLU、LayerNorm 和 Softmax。它对应 Transformer non-linear modules：上层算子本身不是新的信任边界，而是调用 SMP、ODReLU、base/plus hybrid approximation 等底层安全原语。

GeLU 依赖 Erf 或 Sigmoid 类近似，LayerNorm 的瓶颈是方差上的 InvSqrt+，Softmax 的瓶颈是 Exp、行求和后的 Inv+ 以及最终归一化乘法。所有输入和输出均保持为二方秘密分享，通信中只出现掩码消息。

该工具集的定位高于 basic_protocols 和 hybrid_approx：它不再单独测试某个数学函数，而是把多个安全原语组合成 Transformer 中实际使用的非线性层。设计目标是把线性归约、常数乘加等低成本部分留在本地，把真正需要交互的非线性近似和乘法压缩到少量安全协议调用中。

1.1 模块定位

nonlinear_functions 是最接近模型算子的测试层。basic_protocols 验证 SMP、3SMP、ODReLU、ReLU 等底层原语；base_hybrid_approx 和 plus_hybrid_approx 验证单个数学函数近似；本模块验证这些原语组合后，是否能够完成 Transformer 中真实出现的 GeLU、LayerNorm 和 Softmax。

这三个算子代表三类不同的非线性组合。GeLU 是“近似函数 + 一次安全乘法”；LayerNorm 是“行归约 + 方差 + InvSqrt+ + 仿射变换”；Softmax 是“裁剪/指数 + 行求和 + Inv+ + 归一化乘法”。因此本模块不只关注某个函数的平均误差，还要关注误差在上层公式中的放大方式、每行共享中间量的影响，以及通信轮次在向量长度上如何累积。

算子	主要底层依赖	主要误差来源	主要性能瓶颈
GeLU	Erf/Erf+、SMP	Erf 近似误差被 x 放大	每个元素一次 Erf 和一次乘法
LayerNorm	SMP/Square、InvSqrt+	方差和 inv_std 误差影响整行	平方、InvSqrt+、逐元素归一化乘法
Softmax	CMP/clip、Exp/Exp+、Inv+、SMP	clipping、Exp、Inv+ 共同影响概率分布	每元素 Exp 和最终归一化乘法

1.2 系统参与方与安全边界

本模块仍然使用二方加性分享和 TEE 辅助协议。CSP0/CSP1 持有输入 tensor 的分享，REE/App 调度 GeLU、LayerNorm、Softmax 的执行路径，TEE/Enclave 提供底层协议需要的掩码、比较辅助和近似辅助材料。

上层算子会产生更多有意义的中间统计量，例如 LayerNorm 的 `mean`、`var`、`inv_std`，Softmax 的 clipping gate、指数值 e_i 和行和 s。这些量虽然不是原始输入，但仍然包含输入分布信息，因此不能在在线阶段公开。协议允许在分享域中本地求和、线性组合和调用安全乘法，但不允许打开行统计量、概率分布或非线性函数中间值。

1.3 与 base/plus 协议的关系

GeLU、LayerNorm 和 Softmax 并不重新定义底层近似协议，而是选择合适的 base 或 plus 函数作为子模块。若输入范围已经受控，可以调用 base approximation；若输入分布较宽或存在尾部风险，应调用 plus approximation。实际实现中，LayerNorm 和 Softmax 通常更依赖 plus 版本，因为方差、行和、logits 分布都可能跨越较宽范围。

这种分层让调试更清楚。若单个 Erf+/InvSqrt+/Exp+ 的误差已经异常，应先回到 hybrid approximation 层检查参数；若单函数误差正常，但 GeLU/LayerNorm/Softmax 误差变大，则应检查上层组合公式、缩放回调、逐元素乘法和行级归约是否一致。

2. 逻辑推导

2.1 GeLU

GeLU 使用 Erf 表达：`GeLU(x) = 0.5 * x * (1 + erf(x / sqrt(2))) = 0.5*x + 0.5*x*erf(x / sqrt(2))`。第一项是本地线性计算，第二项调用 Erf 近似和 SMP。

在分享域中，双方先本地计算 $\langle u\rangle_i = \langle x\rangle_i / \mathrm{sqrt}(2)$，然后调用 `Erf/Erf+` 得到 $\langle e\rangle_i \sim= \langle \mathrm{erf}(u)\rangle_i$，最后调用安全乘法得到 $\langle m\rangle_i = \mathrm{SMP}(\langle x\rangle_i, \langle e\rangle_i)$，并输出 $\langle \mathrm{GeLU}(x)\rangle_i = 0.5*\langle x\rangle_i + 0.5*\langle m\rangle_i$。

从协议角度看，GeLU 的线性部分不需要交互，真正的交互集中在 Erf/Erf+ 和 x*e。如果使用 Erf+，协议会先在误差函数内部完成区间 gate 和饱和选择；GeLU 层只接收最终的 erf 分享。这样可以避免 GeLU 层重复处理 Erf 的宽范围逻辑。

2.2 LayerNorm

LayerNorm 按行计算：

```
mean = E[x]
var  = E[x²] - E[x]²
y    = (x - mean) * invsqrt(var + eps) * gamma + beta
```

均值和加减常数是线性操作，可在分享上本地完成。平方、归一化乘法、缩放乘法需要 SMP；`invsqrt(var+eps)` 调用 InvSqrt+ 以支持更宽的方差范围。

更细地看，长度为 n 的一行输入先本地得到 $\langle mean\rangle_i = (1/n) * sum_j \langle x_j\rangle_i$。平方均值需要安全平方，即 $\langle E[x^2]\rangle_i = (1/n) * sum_j SMP(\langle x_j\rangle_i, \langle x_j\rangle_i)$；均值平方为 $\langle mean^2\rangle_i = SMP(\langle mean\rangle_i, \langle mean\rangle_i)$。于是：

```
<var>ᵢ = <E[x²]>ᵢ - <mean²>ᵢ
<inv_std>ᵢ = InvSqrt+(<var>ᵢ + eps)
<normⱼ>ᵢ = SMP(<xⱼ>ᵢ - <mean>ᵢ, <inv_std>ᵢ)
<yⱼ>ᵢ = SMP(<normⱼ>ᵢ, <gammaⱼ>ᵢ) + <betaⱼ>ᵢ
```

这里有两个容易混淆的点。第一，`mean` 和 `E[x²]` 的求和本身是线性的，不需要安全乘法；需要交互的是每个 x_j^2 和 $mean^2$。第二，`inv_std` 是一整行共享的缩放因子，它的误差会同时影响该行所有元素，而不是只影响某一个位置。因此 LayerNorm 的误差评估应按行观察最大误差和均值偏移。

2.3 Softmax

Softmax 定义为 $softmax(x_i) = exp(x_i) / sum_j exp(x_j)$。实现时先裁剪输入以避免安全最大值搜索，再用 Exp/Exp+ 计算指数，对行求和后调用 Inv+ 得到倒数，最后通过 SMP 得到归一化结果：

```
eᵢ   = Exp(x_clipᵢ)
s    = sumᵢ eᵢ
inv_s = Inv+(s)
yᵢ   = SMP(eᵢ, inv_s)
```

传统 Softmax 常先计算 `max(x)` 再做 x_i-max(x) 以避免溢出，但安全最大值需要多次比较和通信。这里用 $x_clip_i = clip(x_i, alpha, beta)$ 替代安全最大值搜索，再对 clipped 输入做指数近似。这样牺牲的是极端尾部的精确指数值，但保留了密集区间的主要分布特征，同时减少比较轮次。

Softmax 的行级结构决定了误差会相互耦合。某个元素的 Exp 误差不仅影响自己的分子，也会进入行和 `s`，进而影响整行所有元素的归一化结果。Inv+ 的误差同样是行级误差，因为同一个 `inv_s` 会乘到所有 e_i 上。因此 Softmax 的评估不能只看单个元素的 Exp 精度，还要看整行概率和是否接近 1、最大概率位置是否稳定、尾部概率是否被 clipping 过度压缩。

2.4 组合协议的误差与开销

上层非线性算子的误差不是底层误差的简单相加。GeLU 会把 Erf 误差乘上 `0.5*x`；LayerNorm 会把 InvSqrt+ 误差扩散到整行；Softmax 会通过分母归一化把所有 Exp 误差耦合在一起。因此测试时应同时报告平均误差、最大误差、行级误差和输出分布性质。

开销也需要分层理解。线性求和、常数乘加和分享相减通常是本地开销；SMP/3SMP、ODReLU/CMP、Exp+/Inv+/InvSqrt+ 才带来通信和 ECALL。GeLU 的开销随元素数线性

增长；LayerNorm 的开销同时受行长和 InvSqrt+ 调用次数影响；Softmax 的开销受行长影响最明显，因为每个元素都需要 Exp，最后还需要逐元素乘以同一个倒数。

3. 详细理论解释

Transformer 的非线性模块难点不是单个公式复杂，而是输入范围宽、通信轮次多、比较和最大值搜索昂贵。本工具集包含 GeLU、LayerNorm 和 Softmax 三个上层协议。每个协议都由基础安全原语组合而成，但组合方式、误差来源和性能瓶颈不同。

3.1 共同安全与正确性模型

三个上层协议都遵循同一个组合安全模型。底层协议只输出分享值，上层只对分享值做线性组合或继续调用安全乘法。只要 SMP、ODReLU/CMP、base/plus approximation 各自不泄露输入，顺序组合后的 GeLU、LayerNorm 和 Softmax 也不会额外公开中间明文。

正确性则是近似正确性。重构输出应接近对应明文参考实现，但参考实现需要与协议策略一致：GeLU 应使用同一 Erf/Erf+ 近似目标；LayerNorm 应使用相同 eps、定点缩放和 gamma/beta；Softmax 若使用 clipping，就应与 clipped reference 比较，或者单独说明 clipping 带来的偏差。

3.2 GeLU 协议

GeLU 定义为 `GeLU(x) = 0.5 * x * (1 + erf(x / sqrt(2)))`。协议步骤为：

```
1. 本地线性缩放:        u = x / sqrt(2)
2. 调用 Erf/Erf+:       e = erf(u)
3. 调用 SMP:            m = x * e
4. 本地线性组合:        y = 0.5*x + 0.5*m
```

GeLU 的正确性来自上述公式分解。第一步和第四步是线性操作，可以直接在分享上完成；第二步由 Erf base 或 Erf+ 给出误差函数的分享；第三步由 SMP 给出 x*e 的分享。因此输出分享相加后近似等于 GeLU 明文结果。

GeLU 的误差主要来自 Erf 近似。若 `e_hat = erf(x/sqrt(2)) + epsilon`，则 `GeLU_hat(x) - GeLU(x) ~= 0.5 * x * epsilon`。这说明 |x| 越大，Erf 误差被放大得越明显。因此 GeLU 在宽范围输入下不能只依赖窄区间 Erf base，需要 Erf+ 或分段策略控制尾部误差。GeLU 的安全性由 Erf/Erf+ 和 SMP 的组合安全保证：Erf 输出是分享，SMP 输入也是分享，整个过程中不会公开 x、`erf(x)` 或最终 GeLU 明文。

从工程实现看，GeLU 是三个上层算子中结构最规整的一个。每个元素的计算路径基本独立，因此适合批量化和并行调度。其主要风险不是行级归约，而是定点缩放是否在 `x/sqrt(2)`、Erf 输出和 x*e 之间保持一致。如果 Erf 输出尺度和 SMP 输入尺度不一致，最终 `0.5*x + 0.5*m` 会出现系统性偏移。

3.3 LayerNorm 协议

LayerNorm 对一行向量执行 mean = E[x]、var = E[x²] - E[x]²，并输出 $y_i = (x_i - mean) * invsqrt(var + eps) * gamma_i + beta_i$。

协议步骤为：

```
1. 本地求和得到 mean 分享。
2. 对每个元素调用 SMP/Square 计算 xᵢ²。
3. 本地求和得到 E[x²] 分享。
4. 调用 SMP 计算 mean²。
5. 本地计算 var = E[x²] - mean²。
6. 调用 InvSqrt+ 计算 inv_std = 1/sqrt(var+eps)。
7. 调用 SMP 计算 (xᵢ-mean)*inv_std。
8. 调用 SMP 或本地线性操作完成 gamma/beta 仿射变换。
```

LayerNorm 的正确性来自均值、方差和归一化公式。均值是线性归约，分享可以直接求和；方差中的平方项和 $mean^2$ 需要安全乘法；InvSqrt+ 给出宽范围倒平方根；最后的归一化和缩放仍在分享域完成。

LayerNorm 的误差关键在 inv_std。若 inv_std_hat = inv_std + epsilon，则输出误差近似为 $error_y_i \sim= (x_i - mean) * gamma_i * epsilon$。因为同一个 inv_std 会作用于整行所有元素，所以 InvSqrt+ 的误差会系统性影响整行输出。 reciprocal square root 的输入范围可能很宽，base InvSqrt 在单一区间内不稳定，因此 LayerNorm 需要 InvSqrt+ 的 steep/flat 分段和 Newton refinement。LayerNorm 的安全性来自所有平方、倒平方根和乘法都输出分享，均值和求和只在线性分享上进行，不公开行统计量。

LayerNorm 还需要注意方差的数值稳定性。明文公式 E[x²] - E[x]² 在定点实现中可能出现消减误差，尤其是输入值接近、真实方差较小时。eps 的作用不仅是避免除零，也是限制 1/sqrt(var+eps) 的最大值。若 eps 太小，InvSqrt+ 会频繁落入 steep 区域，误差和开销都会上升；若 eps 太大，归一化强度会被系统性压低。

如果 gamma 和 beta 是公开模型参数，最后一步可以视为公开常数乘加；如果 gamma/beta 也以分享形式输入，则 $norm_i * gamma_i$ 需要 SMP。文档中的流程保留了安全乘法形式，是为了覆盖私有参数或统一测试路径。

3.4 Softmax 协议

Softmax 定义为 $softmax(x_i) = exp(x_i) / sum_j exp(x_j)$。明文实现通常会先减去最大值，即 $softmax(x_i) = exp(x_i - max(x)) / sum_j exp(x_j - max(x))$，但安全求 max(x) 需要大量比较。该协议采用 $x_clip_i = clip(x_i, alpha, beta)$ 替代最大值搜索。

协议步骤为：

```
1. 调用比较协议生成 clipping gate。
```

2. 在分享域构造 x_clip。
3. 调用 Exp/Exp+ 计算 e_i = exp(x_clip_i)。
4. 本地线性求和 s = sum$_i$ e_i。
5. 调用 Inv+ 计算 inv_s = 1/s。
6. 对每个元素调用 SMP: y_i = e_i * inv_s。

Softmax 的正确性是近似正确性：输出等于 clipped 输入上的近似 Softmax。clipping 的作用是避免安全最大值搜索，并把 Exp 近似限制在稳定区间。只要 alpha,beta 覆盖输入分布的密集区域，最终概率分布可以保持较好精度。

Softmax 的误差来自三部分：clipping 改变了尾部输入，Exp/Exp+ 有近似误差，Inv+ 对行和 s 有近似误差。由于 $y_i = e_i * inv_s$，每个 e_i 的误差会影响自己的分子，所有 e_i 的误差还会通过 s 影响整行分母。因此 Softmax 对 Exp 和 Inv+ 的精度都敏感。安全性上，clipping gate 是分享，e_i 是分享，s 是分享，inv_s 是分享，最终归一化乘法也是 SMP 输出分享，因此不会公开 logits、指数值或概率。

Softmax 的另一个重点是行和约束。理想情况下，输出概率满足 sum$_i$ y_i = 1。在定点近似和安全乘法后，行和可能略偏离 1，因此测试时应额外统计 row-sum error。若 row-sum error 明显偏大，通常需要检查 Inv+ 的精度、Exp+ 的缩放和最终 e_i*inv_s 的缩放回调，而不是只看单个元素误差。

clipping 边界需要结合任务输入分布设置。边界过宽会让 Exp+ 近似压力变大，边界过窄会让尾部概率过早变成相近常数，影响注意力分布。对于 attention softmax，最大概率位置和 top-k 概率结构往往比极小尾部概率更重要，因此 clipping 策略应优先保证密集区域和高概率区域的相对顺序。

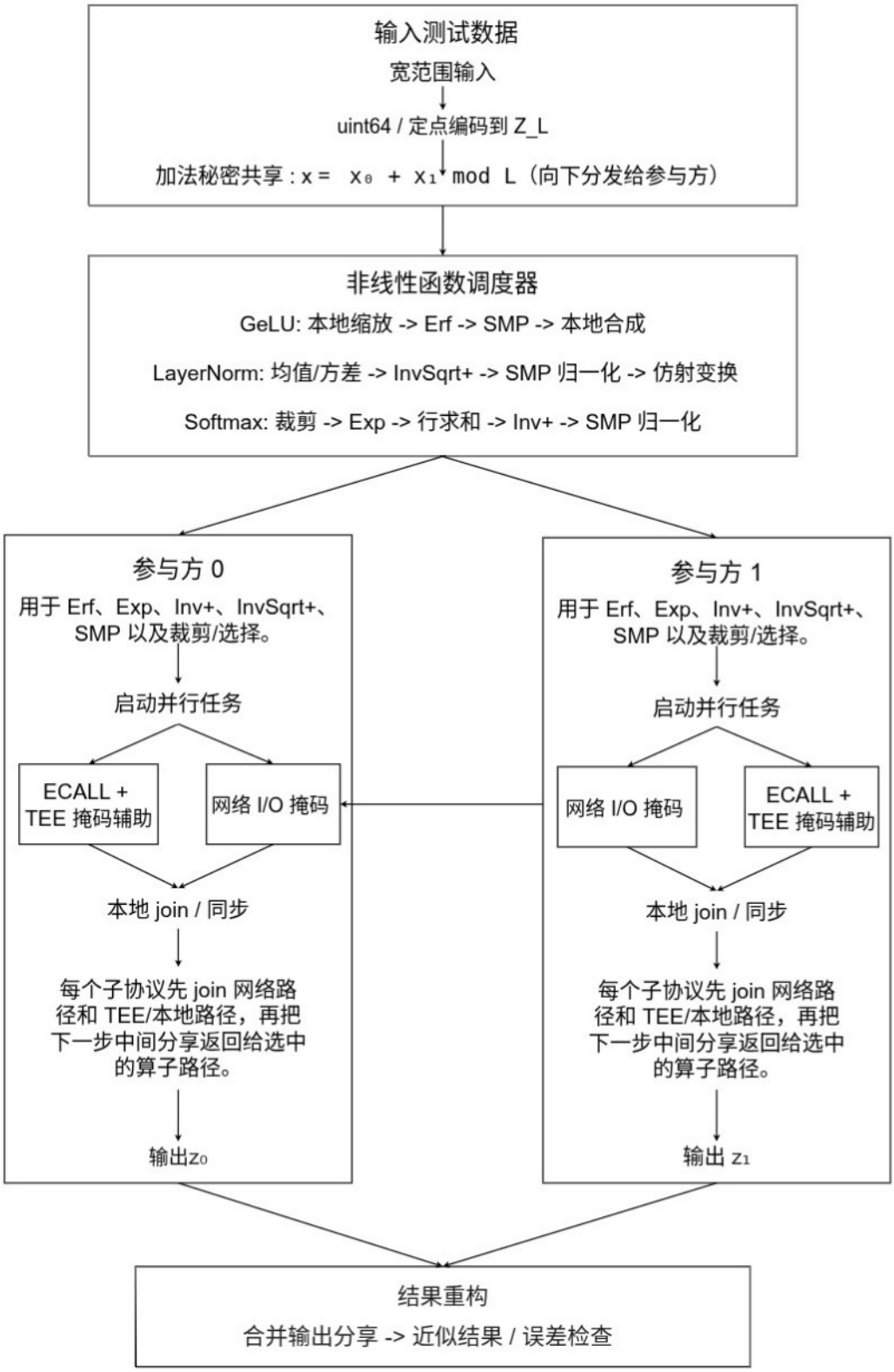
3.5 三个协议的共同实现含义

GeLU、LayerNorm 和 Softmax 都遵循同一组合安全原则：如果 SMP、ODReLU/CMP、base hybrid approximation 和 plus approximation 安全实现各自理想功能，那么这些上层协议只是顺序组合和本地线性计算，不额外泄露中间明文。工程上，GeLU 调用规模大但结构规整，LayerNorm 频繁出现且依赖 InvSqrt+，Softmax 向量长度会放大 Exp 和最终 SMP 调用次数。因此三者的性能瓶颈不同，文档和测试需要分别统计耗时、通信量和轮次。

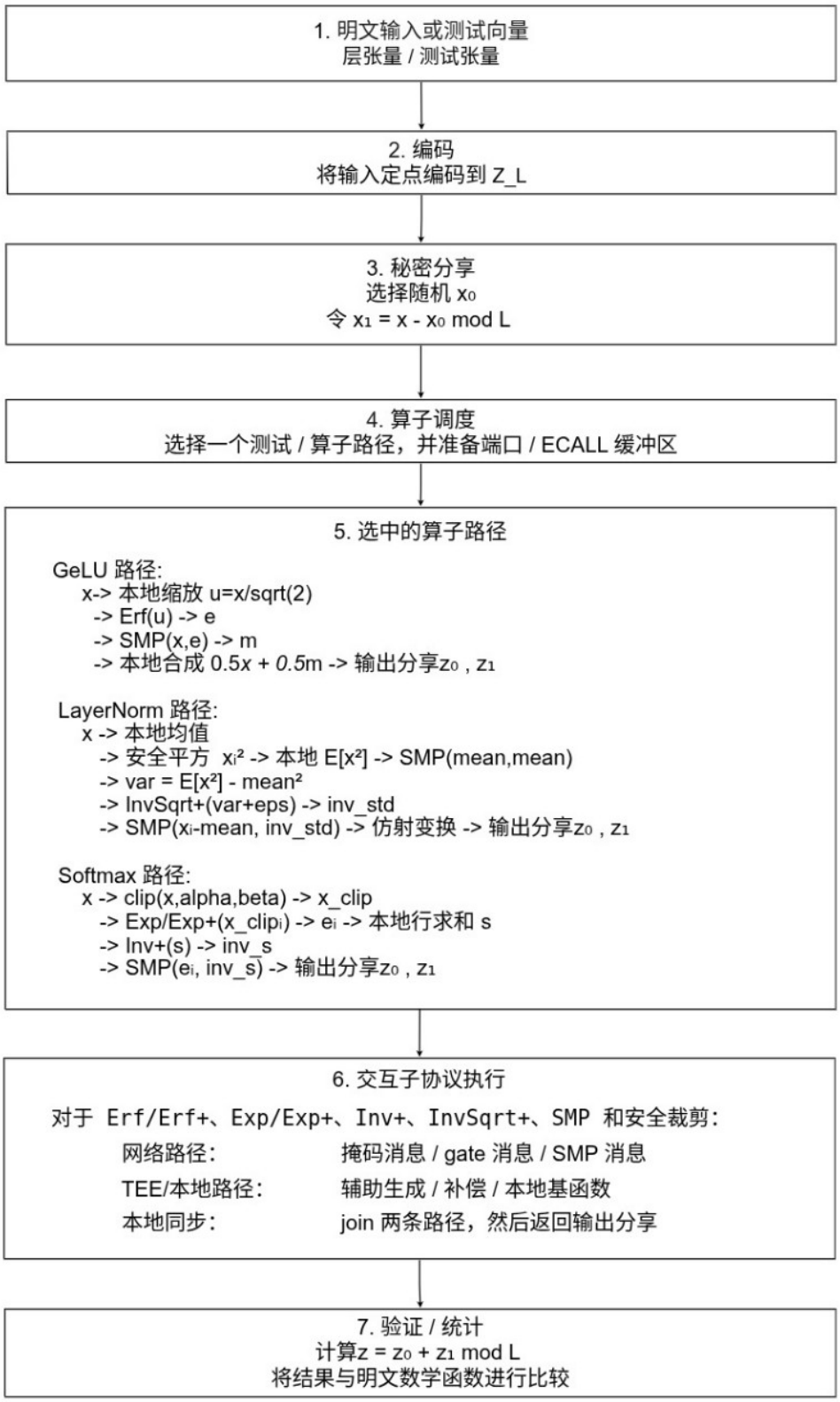
从调试顺序看，应先验证底层单函数近似，再验证上层组合。GeLU 误差异常时优先检查 Erf/Erf+ 和 x*e；LayerNorm 异常时优先检查平方、方差、InvSqrt+ 和 eps；Softmax 异常时优先检查 clipping、Exp+、行和、Inv+ 和 row-sum error。这样可以避免把底层近似参数问题误判为上层调度问题。

从性能优化看，三个协议的优化重点也不同。GeLU 适合批量化 Erf 和 SMP；LayerNorm 适合复用行级 mean、var、inv_std 并减少不必要的逐元素交互；Softmax 适合把 Exp+、行求和和最终归一化乘法组织成流水化批处理。最终报告中应把本地线性计算、ECALL、网络通信和安全乘法调用区分开，否则难以判断瓶颈来自近似函数还是上层组合。

4. 算法框架图


输入测试数据
宽范围输入
uint64 / 定点编码到 Z_L
加法秘密共享 : x = x₀ + x₁ mod L（向下分发给参与方）
非线性函数调度器
GeLU: 本地缩放 -> Erf -> SMP -> 本地合成
LayerNorm: 均值/方差 -> InvSqrt+ -> SMP 归一化 -> 仿射变换
Softmax: 裁剪 -> Exp -> 行求和 -> Inv+ -> SMP 归一化
参与方 0
用于 Erf、Exp、Inv+、InvSqrt+、SMP 以及裁剪/选择。
启动并行任务
ECALL + TEE 掩码辅助
网络 I/O 掩码
本地 join / 同步
每个子协议先 join 网络路径和 TEE/本地路径，再把下一步中间分享返回给选中的算子路径。
输出z₀
参与方 1
用于 Erf、Exp、Inv+、InvSqrt+、SMP 以及裁剪/选择。
启动并行任务
网络 I/O 掩码
ECALL + TEE 掩码辅助
本地 join / 同步
每个子协议先 join 网络路径和 TEE/本地路径，再把下一步中间分享返回给选中的算子路径。
输出 z₁
结果重构
合并输出分享 -> 近似结果 / 误差检查

5. 数据流程图


1. 明文输入或测试向量
层张量 / 测试张量
2. 编码
将输入定点编码到 Z_L
3. 秘密分享
选择随机 x0
令 x1 = x - x0 mod L
4. 算子调度
选择一个测试 / 算子路径，并准备端口 / ECALL 缓冲区
5. 选中的算子路径
GeLU 路径:
x-> 本地缩放 u=x/sqrt(2)
-> Erf(u) -> e
-> SMP(x,e) -> m
-> 本地合成 0.5x + 0.5m -> 输出分享z0 , z1
LayerNorm 路径:
x -> 本地均值
-> 安全平方 xi² -> 本地 E[x²] -> SMP(mean,mean)
-> var = E[x²] - mean²
-> InvSqrt+(var+eps) -> inv_std
-> SMP(xi-mean, inv_std) -> 仿射变换 -> 输出分享z0 , z1
Softmax 路径:
x -> clip(x,alpha,beta) -> x_clip
-> Exp/Exp+(x_clipi) -> ei -> 本地行求和 s
-> Inv+(s) -> inv_s
-> SMP(ei, inv_s) -> 输出分享z0 , z1
6. 交互子协议执行
对于 Erf/Erf+、Exp/Exp+、Inv+、InvSqrt+、SMP 和安全裁剪:
网络路径: 掩码消息 / gate 消息 / SMP 消息
TEE/本地路径: 辅助生成 / 补偿 / 本地基函数
本地同步: join 两条路径，然后返回输出分享
7. 验证 / 统计
计算z = z0 + z1 mod L
将结果与明文数学函数进行比较

6. 运行方式

在 `party0` 和 `party1` 目录分别执行：

```
./1.sh
```

测试宏由 `App/define/define.h` 控制。当前测试输出 GeLU、LayerNorm 和 Softmax 的多配置精度、耗时和通信统计。